

TECHNICAL INTERNSHIP REPORT

AWS DevOps Engineer Internship Assignment

Documentation of Cloud Infrastructure & Web Deployment

Candidate Name:	Galla Harshitha
Email Address:	harshithagalladevops@gmail.com
GitHub Profile:	https://github.com/Harshitha-Galla5
Submission Date:	July 09, 2026

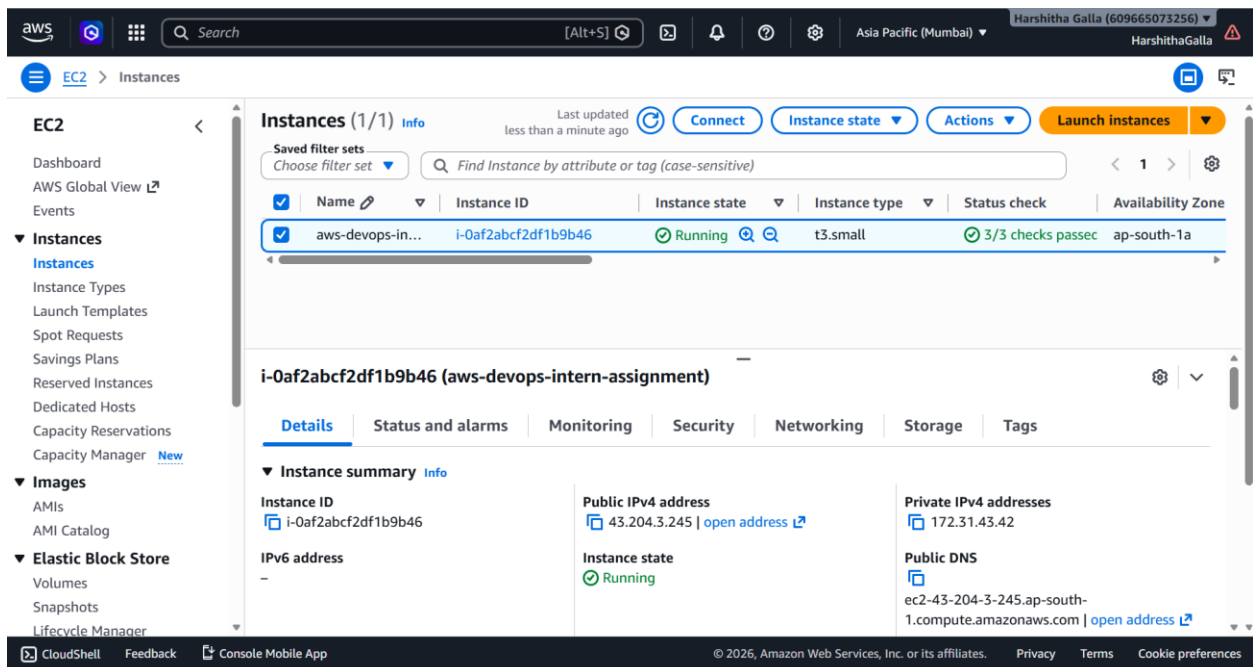
1. AWS Infrastructure Services Used

Amazon EC2 (Elastic Compute Cloud)

Amazon EC2 was utilized to launch a secure Ubuntu Linux virtual machine that served as the foundational hosting platform for this assignment. The EC2 node provided the cloud compute resources required to configure the underlying package sets, manage dependencies, manage server configurations, and expose the live static application via modern networking pipelines.

Key Architectural Roles:

- Hosted the production Ubuntu server instance
- Configured and served the custom static website framework
- Handled basic remote Linux administration and telemetry testing
- Provided isolation vectors via custom Nginx and Docker engine setups

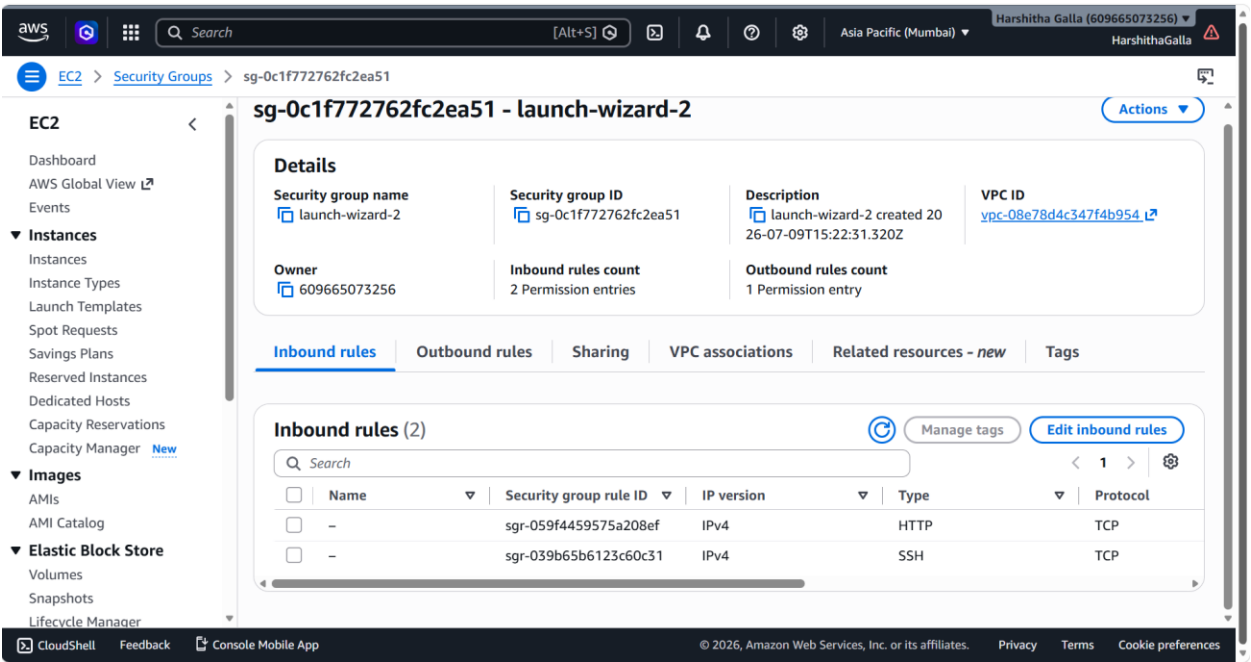


Security Group (Virtual Firewall)

A stateful Security Group was bound to the EC2 compute instance to act as a granular perimeter virtual firewall. This ruleset explicitly dropped unexpected ambient web scans while ensuring necessary application development channels remained transparently reachable.

Configured Rule Standard:

Port Protocol	Inbound Rule Objective	Access Perimeter Scope
SSH (Port 22)	Secures encrypted interactive command-line access for remote development pipelines.	Restricted Administrative DevOps Scope
HTTP (Port 80)	Opens standard web routing access to handle global incoming site user requests.	Globally Public Access (0.0.0.0/0)



Elastic IP Address (Static Networking Bonus Task)

An Elastic IP (EIP) address was allocated inside the VPC environment and associated directly with our operational instance. Because traditional public cloud IPs cycle dynamically following compute reboots, utilizing a persistent Elastic IP secures a static, unyielding entry point that guarantees reliable domain resolutions and uninterrupted user accessibility.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Harshitha Galla (609665073256)

HarshithaGalla

EC2

Elastic IP addresses

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

▼ Load Balancing

Load Balancers

Target Groups

Trust Stores

▼ Auto Scaling

Auto Scaling Groups

Settings

Elastic IP addresses (1/1) Info

Actions

Allocate Elastic IP address

Find elastic IP addresses by attribute or tag

Name

devops-eip

Allocated IPv4 address

43.204.3.245

Type

Public IP

Allocation ID

eipalloc-006593002e39bee9a

43.204.3.245

Allocated IPv4 address

43.204.3.245

Type

Public IP

Allocation ID

eipalloc-006593002e39bee9a

Reverse DNS record

-

Association ID

-

Scope

VPC

Associated instance ID

-

Private IP address

-

Network interface ID

-

Network interface owner account ID

-

Public DNS

-

NAT Gateway ID

-

Address pool

-

Network border group

-

Service managed

-

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

2. Linux System Administration & Implementation Shell Commands

The provisioning, lifecycle validation, and container platform deployment were executed via the standard Linux bash environment. The following implementation table details the sequential logic and execution purposes of the system utilities used:

Terminal Command Run	Technical Operational Objective & System Execution Meaning
<code>sudo apt update</code>	Synchronizes the local package database against source mirrors to fetch the absolute latest updates and patch sheets.
<code>sudo apt upgrade -y</code>	Upgrades all installed server package layers to newer structural dependencies without waiting for interactive prompts.
<code>sudo apt install nginx -y</code>	Fetches, structures, and launches the high-performance Nginx web server engine on the host system.
<code>nginx -v</code>	Exposes the active underlying compiled metadata version of Nginx to cross-reference security compliance benchmarks.
<code>sudo systemctl status nginx</code>	Queries the systemd process manager to explicitly verify that the Nginx daemon service is running stably.
<code>sudo systemctl restart nginx</code>	Flushes active runtime caches and forces an Nginx service cycle to instantly load new configurations and code assets.
<code>df -h</code>	Produces a clear, human-readable summary of mounted storage file volumes to track available storage allocations.
<code>free -h</code>	Exposes system RAM distribution parameters, breaking out active usage channels from buffered spaces.
<code>ps aux</code>	Produces an exhaustive, real-time snapshot summary of all active processes executing across the environment kernels.
<code>sudo apt install docker.io -y</code>	Installs the core upstream containerization infrastructure runtime software components directly on the node.
<code>sudo systemctl enable docker</code>	Registers the primary container runtime daemon to automatically run on instance boot.

sudo systemctl start docker

Fires up the main container hosting platform engine layers on the machine environment instantly.

sudo docker run hello-world

Pulls down and runs a minimalistic verification container asset to guarantee runtime operations are fully functional.

```
ubuntu@ip-172-31-43-42:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-07-09 15:37:14 UTC; 23s ago
     Docs: man:nginx(8)
   Process: 1730 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 1732 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1762 (nginx)
    Tasks: 3 (limit: 2209)
   Memory: 2.4M (peak: 5.1M)
      CPU: 22ms
   CGroup: /system.slice/nginx.service
           └─1762 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─1765 "nginx: worker process"
               └─1766 "nginx: worker process"

Jul 09 15:37:14 ip-172-31-43-42 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server.
Jul 09 15:37:14 ip-172-31-43-42 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
lines 1-17/17 (END)
```

```
ubuntu@ip-172-31-43-42:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.0G  2.0G  4.0G  30% /
tmpfs            954M   0  954M   0% /dev/shm
tmpfs           382M  888K  381M   1% /run
tmpfs            5.0M   0   5.0M   0% /run/lock
eflvars         128K   3.1K  120K   3% /sys/firmware/efl/eflvars
/dev/nvme0n1p16 881M   94M  726M  12% /boot
/dev/nvme0n1p15 105M   6.2M   99M   6% /boot/efl
tmpfs           191M  12K  191M   1% /run/user/1000

ubuntu@ip-172-31-43-42:~$ free -h
             total        used        free      shared  buff/cache   available
Mem:          1.9Gi       383Mi       1.0Gi         2.7Mi       644Mi       1.5Gi
Swap:          0B           0B           0B

ubuntu@ip-172-31-43-42:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.2  0.7 22684 13780 ?        Ss   15:26   0:02 /sbin/init
root         2  0.0  0.0      0     0 ?        S    15:26   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    15:26   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-rcu_gp]
root         5  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-sync_wq]
root         6  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-kvfree_rcu_reclaim]
root         7  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-slab_flushwq]
root         8  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-netns]
root         9  0.0  0.0      0     0 ?        I    15:26   0:00 [kworker/0:0-events]
root        11  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/0:0H-events_highpri]
root        12  0.0  0.0      0     0 ?        I    15:26   0:00 [kworker/u8:0-flush-259:0]
root        13  0.0  0.0      0     0 ?        I<   15:26   0:00 [kworker/R-mm_percpu_wq]
root        14  0.0  0.0      0     0 ?        S    15:26   0:00 [ksoftirqd/0]
root        15  0.0  0.0      0     0 ?        I    15:26   0:00 [rcu_sched]
root        16  0.0  0.0      0     0 ?        S    15:26   0:00 [rcu_exp_par_gp_kthread_worker/0]
root        17  0.0  0.0      0     0 ?        S    15:26   0:00 [rcu_exp_gp_kthread_worker]
root        18  0.0  0.0      0     0 ?        S    15:26   0:00 [migration/0]
root        19  0.0  0.0      0     0 ?        S    15:26   0:00 [idle_inject/0]
root        20  0.0  0.0      0     0 ?        S    15:26   0:00 [cpuhp/0]
root        21  0.0  0.0      0     0 ?        S    15:26   0:00 [cpuhp/1]
```

```
ubuntu@ip-172-31-43-42:~$ sudo systemctl enable docker
ubuntu@ip-172-31-43-42:~$ sudo systemctl start docker
ubuntu@ip-172-31-43-42:~$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-43-42:~$ sudo usermod -aG docker ubuntu
ubuntu@ip-172-31-43-42:~$ newgrp docker
ubuntu@ip-172-31-43-42:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-43-42:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f53e867d88f: pull complete
d5e71e642bf5: Download complete
Digest: sha256:96498ffds22e70807ab6384a5c0485a79b0c7c08ca79ba08623edcad1054e62d
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

ubuntu@ip-172-31-43-42:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	96498ffds22e	25.9kB	9.49kB	10

Info → 10 In Use

3. Practical Engineering Challenges Encountered & Resolutions

Real-world infrastructure configurations rarely execute cleanly without engineering adjustments. The structural friction points found during this deployment cycle were systematically analyzed and mitigated as follows:

- **Security Access Controls & Network Firewalls**

Initially, while ports were being open, verifying explicit network routes was paramount to prevent accidental public blockades. Thorough end-to-end trace validations ensured SSH connections and internet web access routes mapped cleanly to the instance without introducing security vulnerabilities.

- **Nginx Path Mappings & Context Initialization**

Replacing standard default placeholder files at `/var/www/html/` required meticulous root ownership considerations. Elevating folder access through structured administrative command patterns cleanly unlocked deployment spaces without creating permanent permission vulnerabilities across the instance filesystem.

- **Web Daemon Initialization Validation**

Ensuring standard service routing state metrics worked perfectly via `systemd` diagnostics prior to web lookups prevented silent configuration failures, ensuring that config syntax anomalies were caught before traffic hit the server.

- **Docker Core Engine Integration Mapping**

Setting up upstream micro-container runtimes on modern kernels required precise service sequencing. Enabling and testing the container layer using sandbox images proved the setup's integrity before scaling up services.

4. Key Engineering Competencies & Technical Takeaways

This technical internship assignment provided highly impactful exposure to baseline enterprise systems mechanics and infrastructure-as-a-service (IaaS) principles:

- Mastered cloud resource lifecycle provisioning strategies within the Amazon EC2 dashboard framework.
- Established strong security boundaries using stateful firewall ingress/egress security policies.
- Acquired intermediate proficiency in Linux systems administration, folder handling, and diagnostics.
- Engineered production web hosting platforms using high-throughput Nginx reverse-proxy architectures.
- Implemented modern networking standards utilizing AWS Elastic IP blocks for immutable cloud accessibility.

- Explored containerization strategies by managing active system layers using the Docker container ecosystem.
- Maximized professional documentation standards by binding code state controls using Git and GitHub workflows.

← ↻ ⚠ Not secure 43.204.3.245 ☆ ⚙ | 📁 👤 ... 🗨 Chat ▾

AWS DevOps Engineer Internship Assignment

Simple Website Hosted on AWS EC2 using Nginx

Personal Details

Name	Galla Harshitha
College	Dhanekula Institute of Engineering and Technology
Branch	Computer Science Engineering (AI & ML)
Email	harshithagalladevops@gmail.com
Phone	+91 9948670998
Date	09 July 2026

Professional Summary

Computer Science Engineering (AI & ML) student with hands-on experience in AWS, Docker, Kubernetes, Terraform, Jenkins, GitHub Actions, Ansible, Prometheus and Grafana. Passionate about DevOps, cloud infrastructure, automation and continuous learning.

5. Operational Engineering Time Tracking & Conclusion

Time Allocation Analysis

To measure delivery velocities accurately, the lifecycle implementation overhead across the 3 to 4 total hours expended is cataloged below:

Lifecycle Engineering Execution Phase	Time Invested
Launching & provisioning the baseline EC2 node environment	30 Minutes
Securing environment perimeters via custom Security Groups	15 Minutes
Configuring remote interactive sessions via public SSH protocols	10 Minutes
Compiling and fine-tuning the active Nginx web server engine	45 Minutes
Deploying and testing custom static web application assets	30 Minutes
Installing Docker system packages and confirming engine runtime logs	30 Minutes
Allocating, routing, and testing static AWS Elastic IP elements	15 Minutes
Documenting structures and synchronizing files up to GitHub repositories	30 Minutes
Conducting final network security reviews, performance health tests, and telemetry validations	20 Minutes

Assignment Conclusion

This execution exercise cleanly establishes an end-to-end framework for provisioning and managing high-availability web assets inside the Amazon Web Services enterprise cloud ecosystem. Implementing these steps provided valuable exposure to critical cloud architecture, network security design, system administration, and basic container platform mechanics.

By completing both core tasks and advanced bonus options—such as static network route bindings with Elastic IPs and localized container verification cycles with Docker—the overall deployment strategy demonstrates standard operational industry practices.